

EJEMPLAR DE LA OBRA

Registro de Programa de Cómputo — INDAUTOR

SMARTUR: Motor Híbrido de Recomendación Turística

Tipo de obra: Programa de cómputo

Autores

Martín Lara Olivares

Fernanda Pacheco Banda

Aarón Ochoa Ramírez

Universidad Tecnológica del Centro de Veracruz — Cuitláhuac, Veracruz, México

Contenido: código fuente del motor de recomendación (MODELO).
Este ejemplar contiene las primeras 10 y las últimas 10 páginas
del listado de código fuente, conforme al requisito de INDAUTOR.

Cuitláhuac, Veracruz — 05/08/2026

```

===== MODELO/src/fusion.py =====
1 """
2 SMARTUR Fusion v4: Pipeline de dos etapas (Retrieval -> Ranking).
3
4 Fase A (Retrieval):
5 1. Pool amplio de ~200 candidatos vía KNN/Pearson
6 2. Filtro duro: elimina ítems que violen restricciones binarias
7 3. Filtro suave: prioriza categorías según tiposTurismo
8
9 Fase B (Ranking) – 4 señales:
10 - CF/SVD Pearson (warm users only)
11 - RF contextual [Item + User + Match features]
12 - LightFM WARP (cold-start + warm via feature embeddings)
13 - ContentModel TF-IDF (fallback cuando LightFM no disponible)
14
15 Blending (RF de-ponderado – el peor estimador en datos reales y sintéticos;
16 ver constantes LFM_W_* / RF_W_* más abajo):
17 Cold-start: 0.70 LightFM + 0.30 RF
18 Warm user: 0.40 LightFM + 0.35 CF + 0.25 RF
19 """
20
21 import numpy as np
22 import pandas as pd
23
24 from cf import REAL_SIGNAL_SOURCES, predict_cf_pearson_with_source
25 from context_encoder import MAPEO_CATEGORIAS
26 from poi_repository import fetch_all_items
27
28 # Center of Altas Montañas region – used as user location fallback
29 _ALTAS_MONTANAS_LAT = 18.95
30 _ALTAS_MONTANAS_LON = -97.05
31
32 # — Blend dinámico por densidad de datos reales (data_warmth ∈ [0,1]) —————
33 # El peso de la preferencia declarada (contenido) NO es fijo: depende de cuántos
34 # datos reales tiene el sistema. Con catálogo/uso frío (data_warmth=0), la
35 # preferencia DOMINA – es la única señal que discrimina cuando CF/RF aún no
36 # aprendieron nada (todos devuelven un valor plano). Conforme se acumulan
37 # interacciones reales (data_warmth=1), la preferencia baja a su peso "maduro"
38 # y los modelos aprendidos (CF/RF/LightFM) toman el control. Todo automático,
39 # sin ajustar pesos a mano.
40 # pref_weight = COLD - (COLD - WARM) * data_warmth
41 PREF_WEIGHT_COLD = 0.65 # frío: la preferencia manda
42 PREF_WEIGHT_WARM = 0.20 # maduro: peso "de fondo" (era el valor fijo anterior)
43
44 # Retrocompatibilidad: algunos módulos/tests importan esta constante. Equivale
45 # al peso maduro (warm). El valor efectivo lo calcula _preference_weight().
46 PREFERENCE_BOOST_WEIGHT = PREF_WEIGHT_WARM
47
48
49 # — Pesos del blend de servicio (bloque de modelos aprendidos) —————
50 # El Random Forest es, de forma CONSISTENTE, el peor estimador de rating: tanto
51 # en datos reales (RMSE 2.227 vs item_mean 0.883) como en el experimento
52 # sintético con estructura latente aprendible a propósito (item_mean 1.067 <
53 # CF 1.174 < RF 1.603, ver synthetic_persona_validation.py). Ni siquiera cuando
54 # hay estructura que aprender el RF/CF superan a un simple promedio. Por eso su
55 # peso en la recomendación SERVIDA se redujo a favor de LightFM (basado en
56 # features, el mejor en arranque en frío) y CF. Los tres modelos siguen
57 # construyéndose, entrenándose, evaluándose y comparándose en el dashboard
58 # (requisito académico intacto); esto solo ajusta cuánto influye cada uno en el
59 # score final que ve el usuario.

```

```

60 LFM_W_COLD, RF_W_COLD          = 0.70, 0.30      # antes 0.60 / 0.40
61 LFM_W_WARM, CF_W_WARM, RF_W_WARM = 0.40, 0.35, 0.25 # antes 0.30 / 0.30 / 0.40
62 RF_W_MAX_NOLFM                 = 0.50           # tope al RF sin LightFM (antes ~0.94)
63
64
65 def _preference_weight(data_warmth: float) -> float:
66     """Peso de la preferencia en el blend final según data_warmth ∈ [0,1].
67     Interpola linealmente entre PREF_WEIGHT_COLD (frío) y PREF_WEIGHT_WARM
68     (maduro). Fuera de rango se recorta a [0,1]."""
69     w = min(1.0, max(0.0, float(data_warmth)))
70     return PREF_WEIGHT_COLD - (PREF_WEIGHT_COLD - PREF_WEIGHT_WARM) * w
71
72 # presupuesto_bucket (fetch_traveler_profile, poi_repository.py) es un string;
73 # price_level en df_biz es numérico (misma escala 1-4 que usa rf_model.py para
74 # user_budget). Se mapea aquí para poder comparar ambos.
75 _BUDGET_BUCKET_LEVEL = {'bajo': 1, 'medio': 2, 'alto': 3, 'premium': 4}
76
77 # Keywords per tourism type for explanation tags
78 _TOURISM_KEYWORDS: dict[str, list[str]] = {
79     'naturaleza': ['park', 'hiking', 'waterfall', 'nature', 'botanical', 'outdoor', 'mountain',
80                   'volcano'],
81     'aventura': ['adventure', 'hiking', 'mountain', 'volcano', 'outdoor'],
82     'cultural': ['museum', 'history', 'monument', 'landmark', 'cathedral', 'market'],
83     'gastronomico': ['restaurant', 'food', 'gastronomy', 'cafe', 'market'],
84     'rural': ['hacienda', 'ranch', 'rural', 'countryside', 'sanctuary'],
85     'nocturno': ['bar', 'pub', 'nightlife', 'cantina', 'lounge', 'antro'],
86 }
87
88 def _compute_dist_km_series(df: pd.DataFrame, user_lat: float, user_lon: float) -> pd.Series:
89     """Haversine distance (km) from (user_lat, user_lon) to each row in df."""
90     R = 6371.0
91     lat1 = np.radians(user_lat)
92     lon1 = np.radians(user_lon)
93     lat2 = np.radians(df['latitude'].fillna(user_lat))
94     lon2 = np.radians(df['longitude'].fillna(user_lon))
95     dlat = lat2 - lat1
96     dlon = lon2 - lon1
97     a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
98     return (2 * R * np.arcsin(np.sqrt(a))).clip(0, 500)
99
100
101 def _build_reason_tags(
102     biz_id: str,
103     cats: str,
104     context: dict,
105     quality_scores: dict | None,
106     dist_km: float | None,
107 ) -> list[str]:
108     """
109     Generates up to 2 human-readable explanation tags for a recommendation:
110     1. Tourism-type match (e.g. "Coincide con naturaleza")
111     2. Quality signal (e.g. "Muy bien evaluado") - OR -
112     Distance signal (e.g. "A 3.2 km de ti")
113     """
114     tags: list[str] = []
115     cats_lower = cats.lower() if cats else ''
116     tipos: list[str] = context.get('tiposTurismo', []) or []
117
118     # Tourism match

```

```

119     for tipo in tipos:
120         keywords = _TOURISM_KEYWORDS.get(tipo, [])
121         if any(k in cats_lower for k in keywords):
122             tags.append(f"Coincide con {tipo}")
123             break
124
125     if len(tags) < 2:
126         # Quality signal (priority over distance when service is well evaluated)
127         q = (quality_scores or {}).get(biz_id, 0.0)
128         if q >= 0.7:
129             tags.append("Muy bien evaluado")
130         elif q >= 0.5:
131             tags.append("Bien evaluado")
132         # Distance signal (fallback when no quality data)
133         elif dist_km is not None:
134             if dist_km < 5:
135                 tags.append(f"A {dist_km:.1f} km de ti")
136             elif dist_km < 15:
137                 tags.append(f"Cerca ({dist_km:.0f} km)")
138
139     return tags[:2]
140
141 # -----
142 # Filtros
143 # -----
144
145 def filtro_duro(biz_df, context):
146     """
147     Elimina candidatos que violen restricciones binarias del usuario.
148     Son condiciones de 'no-negociable' (deal-breakers).
149     """
150     if not context:
151         return biz_df
152
153     filtered = biz_df.copy()
154
155     # Accesibilidad: si el usuario la requiere, eliminar negocios sin ella
156     if context.get('requiere_accesibilidad') is True:
157         if 'is_accessible' in filtered.columns:
158             filtered = filtered[filtered['is_accessible'] == 1]
159
160     # Outdoor: si el usuario lo prefiere como requisito duro
161     if context.get('pref_outdoor') is True:
162         if 'outdoor' in filtered.columns:
163             filtered = filtered[filtered['outdoor'] == 1]
164
165     # Hotel: si el usuario lo necesita como base
166     if context.get('needs_hotel') is True:
167         mask = filtered['categories'].str.contains('Hotels & Travel|Hotels', case=False, na=False)
168         filtered = filtered[mask]
169
170     # Comida: si el usuario NO quiere comida
171     if context.get('pref_food') is False:
172         mask = ~filtered['categories'].str.contains('Restaurants|Food', case=False, na=False)
173         filtered = filtered[mask]
174
175     return filtered
176
177
178 def _diversify(recs, biz_cat_lookup, top_n, max_per_main_cat=2):

```

```

179     """
180     Re-rankea la lista ordenada por score para que el top-N tenga
181     máximo max_per_main_cat ítems con la misma categoría principal.
182     Mantiene el orden de relevancia y solo desplaza los duplicados al final.
183     """
184     cat_counts = {}
185     selected = []
186     overflow = []
187
188     for rec in recs:
189         cats_str = str(biz_cat_lookup.get(rec['item_id'], ''))
190         main_cat = cats_str.split(',')[0].strip() if cats_str else 'Unknown'
191         if cat_counts.get(main_cat, 0) < max_per_main_cat:
192             selected.append(rec)
193             cat_counts[main_cat] = cat_counts.get(main_cat, 0) + 1
194         else:
195             overflow.append(rec)
196             if len(selected) >= top_n:
197                 break
198
199     if len(selected) < top_n:
200         selected.extend(overflow[:top_n - len(selected)])
201     return selected[:top_n]
202
203
204 def _categorias_para_tipos(tipos: list[str]) -> list[str]:
205     """Expande una lista de tiposTurismo a sus palabras clave de categoría
206     (MAPEO_CATEGORIAS) – compartido entre el filtro suave (legado) y el
207     boost de preferencia."""
208     categorias: list[str] = []
209     for tipo in tipos or []:
210         categorias.extend(MAPEO_CATEGORIAS.get(tipo, []))
211     return categorias
212
213
214 def filtrar_candidatos_por_contexto(biz_df, context):
215     """Filtro suave: prioriza negocios cuyas categorías coincidan con tiposTurismo."""
216     if not context:
217         return biz_df
218
219     filtered_df = biz_df.copy()
220
221     if 'tiposTurismo' in context and context['tiposTurismo']:
222         yelp_categories = _categorias_para_tipos(context['tiposTurismo'])
223         if yelp_categories:
224             mask = filtered_df['categories'].str.contains(
225                 '|'.join(yelp_categories), case=False, na=False
226             )
227             # Si ningún candidato coincide, mantener todos (degradación graciosa)
228             if mask.any():
229                 filtered_df = filtered_df[mask]
230
231     return filtered_df
232
233
234 # Categorías que cuentan como "turismo sostenible" – mismo criterio que ya
235 # se usa para tiposTurismo naturaleza/rural (áreas naturales, comunidades
236 # rurales), no hay una bandera dedicada de sostenibilidad a nivel de ítem.
237 _SUSTAINABLE_TIPOS = ['naturaleza', 'rural']
238

```

```

239
240 def preference_match_score(
241     categories: str,
242     context: dict | None,
243     price_level: float | None = None,
244     is_romantic: int = 0,
245     is_good_for_kids: int = 0,
246     outdoor: int = 0,
247 ) -> float:
248     """
249     Qué tan bien encaja un candidato con las preferencias DECLARADAS por el
250     usuario (perfil real), independiente de lo que CF/RF hayan aprendido de
251     calificaciones. Devuelve un valor en [0, 1]:
252     - 0.45 si la categoría coincide con algún tiposTurismo declarado
253     - 0.20 según qué tan cerca esté price_level del presupuesto_bucket
254       (penalización gradual, no todo-o-nada)
255     - 0.10 si el tipo de grupo (group_type) coincide con las banderas
256       is_romantic/is_good_for_kids del ítem
257     - 0.15 si preferred_place ('aire'/'cerrado') coincide con el flag
258       outdoor del ítem
259     - 0.10 si sustainable_preferences está activo y la categoría es de
260       naturaleza/rural
261
262     Estas dos últimas señales ('preferred_place', 'sustainable_preferences')
263     ya se le preguntan al usuario en el perfil (traveler_profile) y se
264     guardan en la BD, pero antes nunca llegaban al motor de recomendación
265     – se pedían y se ignoraban.
266
267     A diferencia de filtrar_candidatos_por_contexto (que excluye candidatos
268     y puede degradar en silencio a "sin filtro" si el match da cero), esto
269     nunca elimina candidatos – solo reordena empujando hacia arriba lo que
270     sí coincide, así que la preferencia declarada siempre tiene efecto en el
271     ranking aunque el catálogo de categorías esté mal etiquetado.
272     """
273     if not context:
274         return 0.0
275
276     score = 0.0
277     cats_lower = (categories or '').lower()
278
279     # Tipo de turismo (peso mayor)
280     tipos = context.get('tiposTurismo') or []
281     keywords = _categorias_para_tipos(tipos)
282     if keywords and any(k.lower() in cats_lower for k in keywords):
283         score += 0.45
284
285     # Presupuesto (peso menor, gradual)
286     bucket = context.get('presupuesto_bucket')
287     bucket_level = _BUDGET_BUCKET_LEVEL.get(bucket) if bucket else None
288     if bucket_level is not None and price_level is not None:
289         try:
290             delta = abs(float(price_level) - bucket_level)
291             score += 0.20 * max(0.0, 1.0 - delta / 3.0)
292         except (TypeError, ValueError):
293             pass
294
295     # Tipo de grupo (peso menor)
296     group_type = (context.get('group_type') or '').lower()
297     if group_type == 'familia' and is_good_for_kids:
298         score += 0.10

```

```

299     elif group_type == 'pareja' and is_romantic:
300         score += 0.10
301
302     # Lugar preferido (aire libre / cerrado) – 'indiferente' no suma ni resta.
303     preferred_place = (context.get('preferred_place') or 'indiferente').lower()
304     if preferred_place == 'aire' and outdoor:
305         score += 0.15
306     elif preferred_place == 'cerrado' and not outdoor:
307         score += 0.15
308
309     # Preferencia de turismo sostenible
310     if context.get('sustainable_preferences'):
311         sustainable_keywords = _categorias_para_tipos(_SUSTAINABLE_TIPOS)
312         if sustainable_keywords and any(k.lower() in cats_lower for k in sustainable_keywords):
313             score += 0.10
314
315     return min(1.0, score)
316
317
318 def _resolve_cf_score(
319     user_id,
320     biz_id,
321     engine,
322     *,
323     matrix_col_set: set,
324     local_id_set: set,
325     quality_scores: dict | None,
326     global_mean_rating: float,
327 ) -> tuple[float, str]:
328     """
329     Decide la señal colaborativa de un candidato y de dónde salió.
330
331     ORDEN IMPORTANTE – aquí vivía un bug estructural: antes se preguntaba
332     PRIMERO 'if biz_id in local_id_set' (y se usaba el proxy de calidad).
333     Como en producción TODOS los candidatos son POIs locales, esa rama
334     siempre ganaba y 'predict_cf_pearson' NUNCA se ejecutaba para un lugar
335     recomendado – el CF-KNN estaba muerto y no podía despertar por más
336     interacciones reales que se acumularan.
337
338     Ahora se intenta el CF SIEMPRE primero, y el proxy de calidad solo entra
339     cuando el CF no tiene señal real (devuelve 'fallback_mean', o sea un
340     promedio sin información). Así el CF-KNN se activa solo en cuanto haya
341     interacciones reales sobre POIs locales: los IDs ya coinciden (poi_N /
342     svc_N tanto en fetch_all_items como en fetch_real_interactions) y esas
343     interacciones ya se mezclan al entrenamiento del engine.
344
345     Returns:
346         (score, signal) donde signal ∈ {'knn', 'svd', 'quality_proxy', 'global_mean'}
347         – 'knn'/'svd' significan que el CF realmente aportó.
348     """
349     if engine is not None and biz_id in matrix_col_set:
350         cf_pred, cf_source = predict_cf_pearson_with_source(user_id, biz_id, engine)
351         if cf_source in REAL_SIGNAL_SOURCES:
352             return cf_pred, cf_source
353
354     # Sin señal CF real todavía.
355     if biz_id in local_id_set:
356         # Proxy: evaluación de calidad del admin (service_evaluation) – así al
357         # menos los POIs locales se diferencian entre sí en vez de compartir
358         # un score plano.

```

```

359         q = (quality_scores or {}).get(biz_id)
360         if q is not None:
361             return 3.0 + 2.0 * q, 'quality_proxy'
362
363     return global_mean_rating, 'global_mean'
364
365
366 # -----
367 # Pipeline principal
368 # -----
369
370 def recommend_hybrid(
371     user_id, engine, context_model,
372     alpha=0.4, context=None, top_n=5,
373     lightfm_model=None, content_model=None,
374     quality_scores=None, data_warmth=0.0,
375 ):
376     """
377     Sistema Híbrido SMARTUR v4 (Retrieval + Ranking multi-señal).
378
379     Modo producción (local POIs disponibles):
380     - Candidatos: SOLO POIs de la BD local (Altas Montañas, Veracruz)
381     - Ranking: LightFM + RF (cold-start); LightFM + CF + RF (warm)
382
383     Modo desarrollo/fallback (sin BD local):
384     - Candidatos: 200 negocios Yelp vía KNN
385     - Ranking:  $\alpha \times CF + (1-\alpha) \times RF$  (+ LightFM si disponible)
386     """
387     # — Fase A: Retrieval -----
388     try:
389         local_pois = fetch_all_items()
390         local_biz = context_model.prepare_local_items(local_pois)
391     except Exception:
392         local_biz = pd.DataFrame()
393
394     if not local_biz.empty:
395         # Producción: POIs locales como pool exclusivo.
396         # Usamos un alpha reducido (no cero) para que la señal latente de CF/SVD
397         # siga contribuyendo al score final, especialmente con el fallback SVD.
398         local_biz = context_model.add_category_features(local_biz)
399         biz_candidates = local_biz
400         try:
401             from model_metrics import load_metrics as _lm
402             _saved_alpha = float(_lm().get('best_alpha', 0.2))
403             # Reducir alpha a 30% del óptimo global para dar más peso al RF contextual
404             # que sí fue entrenado con features de los POIs locales.
405             effective_alpha = max(0.05, _saved_alpha * 0.3)
406         except Exception:
407             effective_alpha = 0.1 # fallback conservador
408     else:
409         # Fallback desarrollo: candidatos Yelp KNN
410         if engine is None:
411             return [] # no local POIs and no engine -> cannot recommend
412         candidate_ids = engine.get_candidate_pool(user_id, top_n=200)
413         biz_candidates = engine.df_biz[engine.df_biz['business_id'].isin(candidate_ids)]
414         effective_alpha = alpha
415
416     # Filtro duro (restricciones binarias – deal-breakers reales, ej. accesibilidad)
417     refined_df = filtro_duro(biz_candidates, context)
418

```

```

419 # Fallback: si el filtro duro vació la lista, usar pool sin filtrar
420 if refined_df.empty:
421     refined_df = biz_candidates
422
423 # NOTA: el filtro suave por tiposTurismo (filtrar_candidatos_por_contexto)
424 # ya NO se aplica aquí – excluía candidatos por texto libre y podía
425 # degradar en silencio a "sin filtro" si el match daba cero resultados
426 # (categories_raw/categories_mapped son texto sin validación fuerte).
427 # En su lugar, la preferencia declarada se usa como boost de score más
428 # abajo (preference_match_score) – nunca excluye, siempre tiene efecto.
429
430 final_ids = refined_df['business_id'].tolist()
431
432 # — Fase B: Ranking —————
433 ref_df = local_biz if not local_biz.empty else (engine.df_biz if engine is not None else pd.DataFrame())
434 rf_scores = context_model.predict_with_context(final_ids, user_context=context, df_biz_override=ref_df)
435 rf_map = dict(zip(final_ids, rf_scores))
436
437 biz_cat_lookup = ref_df.set_index('business_id')['categories'].to_dict()
438 all_biz_names = ref_df.set_index('business_id')['name'].to_dict()
439 biz_kind_lookup = ref_df.set_index('business_id')['kind'].to_dict() if 'kind' in ref_df.columns else {}
440 biz_desc_lookup = ref_df.set_index('business_id')['description'].to_dict() if 'description' in ref_df.columns else {}
441 local_id_set = set(local_biz['business_id']) if not local_biz.empty else set()
442 matrix_col_set = set(engine.user_item_matrix_columns) if engine is not None else set()
443
444 # Lookups para el boost de preferencia (preference_match_score)
445 price_lookup = ref_df.set_index('business_id')['price_level'].to_dict() if 'price_level' in ref_df.columns else {}
446 is_romantic_lookup = ref_df.set_index('business_id')['is_romantic'].to_dict() if 'is_romantic' in ref_df.columns else {}
447 is_kids_lookup = ref_df.set_index('business_id')['is_good_for_kids'].to_dict() if 'is_good_for_kids' in ref_df.columns else {}
448 outdoor_lookup = ref_df.set_index('business_id')['outdoor'].to_dict() if 'outdoor' in ref_df.columns else {}
449
450 # — Distance map for explanation tags —————
451 # Use user-supplied lat/lon from context; fall back to region center.
452 dist_km_map: dict[str, float] = {}
453 if not ref_df.empty and 'latitude' in ref_df.columns and 'longitude' in ref_df.columns:
454     try:
455         user_lat = float(context.get('lat') or ALTAS_MONTANAS_LAT)
456         user_lon = float(context.get('lon') or ALTAS_MONTANAS_LON)
457         distances = _compute_dist_km_series(ref_df, user_lat, user_lon)
458         dist_km_map = dict(zip(ref_df['business_id'], distances))
459     except Exception:
460         pass # explanation tags gracefully degrade without distance
461
462 # — LightFM or ContentModel scores —————
463 # Determine cold-start status for blending weights
464 is_cold_start = str(user_id) not in (
465     getattr(lightfm_model, '_known_users', set()) if lightfm_model else set()
466 ) and str(user_id) not in (getattr(engine, 'user_index', {}) or {})
467
468 if lightfm_model is not None and getattr(lightfm_model, 'is_fitted', False):
469     lfm_scores = lightfm_model.predict(str(user_id), final_ids, user_context=context)
470     lfm_map = dict(zip(final_ids, lfm_scores))

```

```

471     use_content = False
472 elif content_model is not None and getattr(content_model, 'is_fitted', False):
473     cb_scores = content_model.score(final_ids, user_context=context)
474     lfm_map = dict(zip(final_ids, cb_scores))
475     use_content = True
476 else:
477     lfm_map = {}
478     use_content = False
479
480 recommendations = []
481
482 global_mean_rating = engine.train_data['stars'].mean() if engine is not None else 3.5
483
484 # Peso dinámico de la preferencia: alto en frío (domina), bajo cuando hay
485 # datos reales suficientes (los modelos aprendidos toman el control).
486 pref_weight = _preference_weight(data_warmth)
487
488 for biz_id in final_ids:
489     # — Señal colaborativa (CF-KNN) con fallback a calidad —————
490     # ORDEN IMPORTANTE. Antes la rama `if biz_id in local_id_set` iba
491     # PRIMERO y siempre ganaba: como en producción todos los candidatos
492     # son POIs locales, predict_cf_pearson NUNCA se ejecutaba para un
493     # lugar recomendado — el CF-KNN estaba estructuralmente muerto y no
494     # podía despertar por más datos reales que se acumularan.
495     #
496     # Ahora se intenta CF primero SIEMPRE, y solo se usa el proxy de
497     # calidad (evaluación del admin) cuando el CF no tiene señal real
498     # (devuelve 'fallback_mean', o sea un promedio sin información).
499     # Así el CF se activa solo en cuanto haya interacciones reales sobre
500     # POIs locales — los IDs ya coinciden (poi_N / svc_N tanto en
501     # fetch_all_items como en fetch_real_interactions), y esas
502     # interacciones ya se mezclan al entrenamiento del engine.
503     score_cf, cf_signal = _resolve_cf_score(
504         user_id, biz_id, engine,
505         matrix_col_set=matrix_col_set,
506         local_id_set=local_id_set,
507         quality_scores=quality_scores,
508         global_mean_rating=global_mean_rating,
509     )
510
511     score_rf = rf_map.get(biz_id, 3.0)
512     score_lfm = lfm_map.get(biz_id, 3.0)
513     cats = biz_cat_lookup.get(biz_id, '')
514
515     # Blending weights (RF de-ponderado — ver constantes arriba):
516     # Cold-start user -> LightFM (feature-based) domina, RF de apoyo
517     # Warm user -> LightFM + CF por delante, RF de apoyo
518     if lfm_map:
519         if is_cold_start:
520             final_score = LFM_W_COLD * score_lfm + RF_W_COLD * score_rf
521         else:
522             final_score = LFM_W_WARM * score_lfm + CF_W_WARM * score_cf + RF_W_WARM * score_rf
523     else:
524         # LightFM not available -> classic blend, pero con tope al RF para no
525         # dejar que el peor estimador domine (antes llegaba a ~0.94).
526         rf_w = min(RF_W_MAX_NOLFM, 1 - effective_alpha)
527         final_score = ((1 - rf_w) * score_cf) + (rf_w * score_rf)
528
529 # Boost de preferencia declarada (perfil real del usuario), no de lo
530 # que CF/RF aprendieron de calificaciones. Nunca excluye candidatos —

```

```

531     # solo empuja hacia arriba lo que coincide con tiposTurismo/presupuesto/
532     # grupo, así que la preferencia siempre influye en el ranking aunque
533     # el modelo no tenga suficientes datos para haberla aprendido solo.
534     score_pref = preference_match_score(
535         cats, context,
536         price_level=price_lookup.get(biz_id),
537         is_romantic=is_romantic_lookup.get(biz_id, 0),
538         is_good_for_kids=is_kids_lookup.get(biz_id, 0),
539         outdoor=outdoor_lookup.get(biz_id, 0),
540     )
541     final_score = (
542         (1 - pref_weight) * final_score
543         + pref_weight * (score_pref * 5.0)
544     )
545
546     # Quality boost from admin service_evaluation (service_evaluation.total_score).
547     # Non-linear: 0 boost for quality ≤ 0.5 (score ≤ 50/100), +0.3 max at quality = 1.0.
548     # This rewards well-evaluated services without demoting unevaluated POIs.
549     if quality_scores:
550         q = quality_scores.get(biz_id, 0.0)
551         if q > 0.5:
552             quality_boost = 0.3 * (q - 0.5) / 0.5 # linear: 0 @ q=0.5, 0.3 @ q=1.0
553             final_score = min(5.0, final_score + quality_boost)
554
555     nombre = all_biz_names.get(biz_id, 'Desconocido')
556     kind = biz_kind_lookup.get(biz_id, 'poi')
557     desc = biz_desc_lookup.get(biz_id, '') or ''
558
559     recommendations.append({
560         'item_id': str(biz_id),
561         'title': str(nombre),
562         'description': str(desc),
563         'category': str(cats),
564         'score': float(round(final_score, 3)),
565         'pred_cf': float(round(score_cf, 3)),
566         # De dónde salió pred_cf: 'knn'/'svd' = CF real aportó;
567         # 'quality_proxy'/'global_mean' = CF aún sin señal (esperando
568         # interacciones reales sobre este POI). Permite ver en /metrics
569         # cuándo el CF-KNN por fin despierta, sin adivinar.
570         'cf_signal': cf_signal,
571         'pred_rf': float(round(score_rf, 3)),
572         'pred_pref': float(round(score_pref, 3)),
573         'kind': kind,
574         'reason_tags': _build_reason_tags(
575             biz_id, cats, context or {},
576             quality_scores, dist_km_map.get(biz_id),
577         ),
578     })
579
580     sorted_recs = sorted(recommendations, key=lambda x: x['score'], reverse=True)
581     return _diversify(sorted_recs, biz_cat_lookup, top_n)

```

```

===== MODELO/src/cf.py =====
1 import numpy as np
2
3 # Contador de qué "camino" toma cada predicción de CF – instrumentación para
4 # medir qué tan seguido el KNN cae en el fallback constante (media global/del
5 # usuario) en vez de predecir con señal real. Con 198K usuarios y 233 items
6 # en el dataset de Yelp, la sospecha es que la mayoría de las predicciones
7 # de evaluación caen en 'fallback_mean', lo que explicaría por qué el

```

[... páginas intermedias omitidas ...]

```

1056         f"[train] Datos reales: {len(real_df)} interacciones, ponderadas {n_re
peats}x"
1057     )
1058     else:
1059         n_real = len(real_df) if real_df is not None else 0
1060         logger.info(f"[train] Solo {n_real} interacciones reales (mínimo {_MIN_REA
L}) - usando solo CSV.")
1061     except Exception as exc:
1062         logger.warning(f"[train] Interacciones reales no disponibles: {exc}")
1063
1064     # — 1b. Merge admin evaluation scores —————
1065     try:
1066         eval_df = fetch_evaluation_scores()
1067         if eval_df is not None and len(eval_df) > 0:
1068             # evaluation scores only cover tourist services (svc_N IDs)
1069             # Merge columns must match training schema
1070             eval_df['user_id'] = eval_df['user_id'].astype(str)
1071             eval_df = eval_df.rename(columns={'business_id': 'business_id', 'stars': '
stars'})
1072             engine.train_data = pd.concat(
1073                 [engine.train_data, eval_df], ignore_index=True
1074             )
1075             logger.info(f"[train] {len(eval_df)} scores de evaluación admin integrados
al entrenamiento.")
1076         except Exception as exc:
1077             logger.warning(f"[train] Scores de evaluación admin no disponibles: {exc}")
1078
1079     # — 1c. Rest-Mex ya incluido en seed_pois_mexico.py —————
1080     restmex_biz = None
1081
1082     # — 2. Rebuild Pearson + SVD matrix —————
1083     logger.info("[train] Actualizando matriz de Pearson + SVD...")
1084     engine.prepare_pearson_matrix()
1085
1086     # — 3. Retrain RF —————
1087     logger.info("[train] Reentrenando Random Forest...")
1088     context_model.train(engine.train_data, df_biz_extra=restmex_biz)
1089
1090     # — 4b. Retrain LightFM —————
1091     # Si LightFM falló al cargar en el boot (pickle incompatible, archivo
1092     # faltante, etc.) queda en None – sin este fallback, el reentrenamiento
1093     # nocturno lo saltaba para siempre porque solo reentrenaba una instancia
1094     # que ya existiera. Se crea una instancia nueva aquí mismo si hace falta,
1095     # así el sistema se autorecupera en el siguiente ciclo de entrenamiento.
1096     if lightfm_model is None:
1097         try:
1098             from lightfm_model import SmarturLightFMModel
1099             lightfm_model = SmarturLightFMModel()
1100             logger.info("[train] LightFM estaba caído – instancia nueva creada para reinte
ntar.")
1101         except Exception as lfm_init_err:
1102             logger.warning(f"[train] No se pudo instanciar LightFM: {lfm_init_err}")
1103             lightfm_model = None
1104
1105     if lightfm_model is not None:
1106         logger.info("[train] Reentrenando LightFM...")
1107         try:
1108             lightfm_model.train(engine.train_data, context_model.df_biz)
1109         except Exception as lfm_err:
1110             logger.warning(f"[train] LightFM reentrenamiento falló: {lfm_err}")

```

```

1111
1112 # — 4c. Refit ContentModel —————
1113 if content_model_cb is not None:
1114     try:
1115         content_model_cb.fit(context_model.df_biz)
1116     except Exception as cm_err:
1117         logger.warning(f"[train] ContentModel refit falló: {cm_err}")
1118
1119 # — 5. Algorithm metrics (RF + CF) —————
1120 logger.info("[train] Calculando métricas de algoritmos...")
1121 metrics = _compute_simple_metrics(engine, context_model)
1122
1123 # Integridad: si el entrenamiento corrió en modo sintético, marcarlo para
1124 # que el dashboard avise que este desempeño es sobre una verdad inventada,
1125 # no sobre turistas reales. Nunca se debe presentar como métrica real.
1126 if _synth_active:
1127     metrics['synthetic_augmented'] = True
1128     _prev_rationale = metrics.get('selection_rationale', '')
1129     metrics['selection_rationale'] = (
1130         "⚠ ENTRENAMIENTO SINTÉTICO (SMARTUR SYNTH_TRAINING activo): estos "
1131         "resultados son sobre personas sintéticas con estructura latente "
1132         "inventada, para validar que el pipeline de ML aprende cuando existe "
1133         "estructura. NO representan el comportamiento de turistas reales. "
1134         + _prev_rationale
1135     )
1136
1137 # — 5b. Enrich with error_dist, data_quality, prediction_distribution —
1138 if 'error_distribution' not in metrics:
1139     try:
1140         enrich = _compute_enrich_from_engine(engine, context_model)
1141         metrics.update(enrich)
1142     except Exception as exc:
1143         logger.warning(f"[train] Enrichment no disponible: {exc}")
1144
1145 # — 6. Ranking metrics (NDCG@5, Precision@5, Hit Rate@10) —————
1146 # Prefer local SMARTUR evaluation when we have enough real interactions;
1147 # Yelp eval always returns NDCG=0 because recommender returns Veracruz POIs.
1148 try:
1149     from evaluate import evaluar_ranking, evaluar_ranking_local
1150     ranking = evaluar_ranking_local(
1151         context_model=context_model,
1152         lightfm_model=lightfm_model,
1153         content_model=content_model_cb,
1154         k=5,
1155     )
1156     _ranking_is_local = False
1157     if ranking is not None:
1158         metrics['ranking'] = ranking
1159         _ranking_is_local = True # evaluación sobre POIs + usuarios reales
1160         logger.info(
1161             f"[train] Ranking local SMARTUR – NDCG@5={ranking.get('ndcg', 0):.4f}, "
1162             f"P@5={ranking.get('precision', 0):.4f}, "
1163             f"HR@10={ranking.get('hit_rate', 0):.4f}, "
1164             f"PrefMatch={ranking.get('preference_match_rate', 0):.4f}"
1165         )
1166     else:
1167         # Fallback: Yelp-based eval (will show NDCG=0 by design)
1168         ranking = evaluar_ranking(engine, context_model, n_users=50, k=5)
1169         if ranking:
1170             metrics['ranking'] = ranking

```

```

1171         logger.info("[train] Ranking Yelp (fallback – sin suficientes datos reales SMA
RTUR)")
1172
1173         # Relevo automático RMSE → NDCG: solo cuando el ranking viene de la
1174         # evaluación LOCAL (POIs + usuarios reales). El fallback Yelp da
1175         # NDCG=0 por diseño y nunca debe convertirse en criterio primario.
1176         if _ranking_is_local:
1177             _apply_ranking_based_selection(metrics)
1178     except Exception as exc:
1179         logger.warning(f"[train] Ranking evaluation omitida: {exc}")
1180
1181     # — 7. Persist locally —————
1182     save_metrics(metrics)
1183     _best_algo = metrics.get('best_algorithm', '-')
1184     _rmse_val = metrics.get('algorithms', {}).get(_best_algo, {}).get('rmse')
1185     _rmse_str = f"{_rmse_val:.4f}" if isinstance(_rmse_val, (int, float)) else '-'
1186     logger.info(f"[train] Métricas guardadas – mejor algoritmo: {_best_algo} (RMSE={_rmse_str}
)")
1187
1188     # — 7b. Append to training history —————
1189     try:
1190         from datetime import datetime, timezone
1191         history_path = Path(MODELS) / "training_history.json"
1192         history = []
1193         if history_path.exists():
1194             with open(history_path) as fh:
1195                 history = json.load(fh)
1196         entry = {
1197             'timestamp': datetime.now(timezone.utc).isoformat(),
1198             'best_algorithm': _best_algo,
1199             'sample_size': metrics.get('sample_size', 0),
1200         }
1201         best_key = metrics.get('best_algorithm', '')
1202         best_rmse = metrics.get('algorithms', {}).get(best_key, {}).get('rmse')
1203         best_mae = metrics.get('algorithms', {}).get(best_key, {}).get('mae')
1204         if best_rmse is not None:
1205             entry['rmse'] = round(float(best_rmse), 4)
1206         if best_mae is not None:
1207             entry['mae'] = round(float(best_mae), 4)
1208         if 'error_distribution' in metrics:
1209             entry['error_within_1'] = metrics['error_distribution'].get('within_1_0', 0)
1210         if 'ranking' in metrics:
1211             entry['ndcg'] = metrics['ranking'].get('ndcg', 0)
1212             entry['hit_rate'] = metrics['ranking'].get('hit_rate', 0)
1213             if 'preference_match_rate' in metrics['ranking']:
1214                 entry['preference_match_rate'] = metrics['ranking']['preference_match_rate']
1215         history.append(entry)
1216         # Keep last 20 entries
1217         history = history[-20:]
1218         with open(history_path, 'w') as fh:
1219             json.dump(history, fh, indent=2, ensure_ascii=False)
1220         logger.info(f"[train] Training history actualizado ({len(history)} entradas)")
1221     except Exception as h_err:
1222         logger.warning(f"[train] No se pudo persistir training history: {h_err}")
1223
1224     # — 8. Persist to DB (only if we have real algorithm data) —————
1225     if not metrics.get('algorithms'):
1226         logger.warning(f"[train] Métricas vacías – no se persiste en DB (datos insuficientes).")
1227
1228     elif _persist_metrics_to_db(metrics):

```

```

1228         logger.info("[train] Métricas persistidas en DB correctamente.")
1229     else:
1230         logger.warning("[train] Métricas solo disponibles en archivo local (DB no actualizada)
1231     .")
1232
1231     # — 9. Refresh quality scores in-memory (new evaluations during training) —
1232     try:
1233         from poi_repository import fetch_quality_scores
1234         new_quality_scores = fetch_quality_scores()
1235         with _models_lock:
1236             quality_scores = new_quality_scores
1237         logger.info(f"[train] Quality scores refrescados: {len(quality_scores)} servicios")
1238     except Exception as exc:
1239         logger.warning(f"[train] Quality scores no actualizados: {exc}")
1240
1241     # Recalcula el warmth: tras entrenar puede haber más interacciones reales.
1242     _refresh_data_warmth()
1243
1244     logger.info(f"[train] Entrenamiento completado. Muestra: {metrics.get('sample_size', '?')}
1245     predicciones.")
1246 except Exception as e:
1247     logger.error(f"[train] Error en background training: {e}", exc_info=True)
1248
1249 @app.post("/train")
1250 def train_full(background_tasks: BackgroundTasks):
1251     """
1252     Inicia el reentrenamiento completo en background (fire-and-forget).
1253     Actualiza la matriz de Pearson y el modelo Random Forest.
1254     Returns immediately with { ok: true }.
1255     """
1256     if engine is None or context_model is None:
1257         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1258     background_tasks.add_task(_run_full_training)
1259     return {"ok": True, "message": "Entrenamiento iniciado en background"}
1260
1261
1262 _cv_in_progress = False # True mientras corre la validación cruzada en background
1263
1264
1265 def _run_cross_validation(k: int = 5, sample_size: int = 3000) -> None:
1266     """
1267     Background worker: k-fold cross-validation de CF/RF/GBM (ver
1268     cross_validation.py). Complementa el train/test split que ya calcula
1269     _run_full_training/_compute_simple_metrics – responde al requisito de
1270     que cada algoritmo se evalúe tanto con validación cruzada como con
1271     división train/test.
1272     """
1273     global _cv_in_progress
1274     _cv_in_progress = True
1275     try:
1276         from cross_validation import run_kfold_cv
1277         logger.info(f"[cv] Iniciando k-fold cross-validation (k={k}, sample_size={sample_size})...
1278     ")
1279     t0 = time.time()
1280     cv_result = run_kfold_cv(engine, k=k, sample_size=sample_size)
1281     cv_result['total execution time_ms'] = (time.time() - t0) * 1000
1282     cv_result['timestamp'] = datetime.now(timezone.utc).isoformat()
1283
1284     cv_path = Path(_MODELS) / "cv_metrics.json"

```

```

1285         with open(cv_path, 'w', encoding='utf-8') as f:
1286             json.dump(cv_result, f, ensure_ascii=False, indent=2)
1287
1288         _persist_cv_metrics_to_db(cv_result)
1289
1290         for algo, m in cv_result['algorithms'].items():
1291             logger.info(
1292                 f"[cv] {algo}: RMSE={m['rmse_mean']:.4f}±{m['rmse_std']:.4f}, "
1293                 f"MAE={m['mae_mean']:.4f}±{m['mae_std']:.4f}, "
1294                 f"tiempo={m['avg_execution_time_ms']:.0f}ms/fold"
1295             )
1296         logger.info(f"[cv] Cross-validation completada en {cv_result['total_execution_time_ms']:.0
1297 f}ms.")
1298     except Exception as exc:
1299         logger.error(f"[cv] Error en cross-validation: {exc}", exc_info=True)
1300     finally:
1301         _cv_in_progress = False
1302
1303 @app.post("/cross-validation")
1304 def cross_validation_endpoint(background_tasks: BackgroundTasks, k: int = 5, sample_size: int = 30
1305 00):
1306     """
1307     Inicia k-fold cross-validation de CF/RF/GBM en background. Guarda el
1308     resultado en models/cv_metrics.json y en la tabla
1309     ml_cross_validation_metrics. Consultar el resultado con GET /cross-validation.
1310     """
1311     if engine is None:
1312         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1313     if _cv_in_progress:
1314         return {"ok": False, "message": "Cross-validation ya en curso."}
1315     background_tasks.add_task(_run_cross_validation, k, sample_size)
1316     return {"ok": True, "message": f"Cross-validation iniciada en background (k={k}, sample_size={
1317 sample_size})"}
1318
1319 @app.get("/cross-validation")
1320 def get_cross_validation():
1321     """Devuelve el último resultado de cross-validation guardado en disco."""
1322     cv_path = Path(_MODELS) / "cv_metrics.json"
1323     if not cv_path.exists():
1324         raise HTTPException(status_code=404, detail="Sin resultados de cross-validation aún. Ejecu
1325 ta POST /cross-validation primero.")
1326     with open(cv_path, 'r', encoding='utf-8') as f:
1327         return json.load(f)
1328
1329 @app.post("/train-rf")
1330 def train_rf():
1331     """
1332     Llamado bajo demanda de re-entrenamiento del Random Forest contextual.
1333     Esto vuelve a generar el árbol utilizando datos actuales de la Base e ignora el archivo local
1334     previo,
1335     creando un fichero `.joblib` en disco en los procesos intermedios que quedará para los futuros
1336     arranques.
1337     """
1338     if engine is None or context_model is None:
1339         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1340     try:
1341         logger.info("Forzando entrenamiento del modelo Random Forest...")

```

```

1339         __, restmex.biz = _get restmex.data()
1340         context.model.train(engine.train_data, df_biz_extra=restmex.biz)
1341         return {"status": "ok", "message": "Modelo entrenado y guardado correctamente."}
1342     except Exception as e:
1343         logger.error(f"Error entrenando modelo: {e}")
1344         raise HTTPException(status_code=500, detail=str(e))
1345
1346
1347 # — Route Optimizer (ACO) —————
1348
1349 class OptimizeStop(BaseModel):
1350     lat: float
1351     lng: float
1352     name: str = ''
1353     duration_minutes: Optional[int] = None    # dwell time at this stop
1354     open_minutes: Optional[int] = None        # opening time in minutes from midnight
1355     close_minutes: Optional[int] = None       # closing time in minutes from midnight
1356
1357 class OptimizeRouteRequest(BaseModel):
1358     stops: List[OptimizeStop]
1359     start_minutes: int = 540 # departure time in minutes from midnight (default 09:00)
1360
1361
1362 @app.post("/optimize-route")
1363 def optimize_route_endpoint(req: OptimizeRouteRequest):
1364     """
1365     Ant Colony Optimization for itinerary stop ordering with time-window support.
1366     Keeps the first stop fixed as departure and finds the shortest open path
1367     through the remaining stops.
1368
1369     Optional per-stop fields: duration_minutes, open_minutes, close_minutes.
1370     Optional request field: start_minutes (default 540 = 09:00).
1371
1372     Returns:
1373         optimized_order (indices into input stops),
1374         original_distance_km, optimized_distance_km, savings_pct.
1375     """
1376     if len(req.stops) < 2:
1377         raise HTTPException(
1378             status_code=422,
1379             detail="Se necesitan al menos 2 paradas para optimizar",
1380         )
1381     stops = [
1382         {
1383             "lat": s.lat,
1384             "lng": s.lng,
1385             "name": s.name,
1386             "duration_minutes": s.duration_minutes,
1387             "open_minutes": s.open_minutes,
1388             "close_minutes": s.close_minutes,
1389         }
1390         for s in req.stops
1391     ]
1392     result = _aco_optimize(stops, start_minutes=req.start_minutes)
1393     return result
1394
1395
1396 # — WellTur – Wellness Tourism Endpoints —————
1397
1398

```

```

1399 class WellnessAssessmentRequest(BaseModel):
1400     q1: int = Field(..., ge=1, le=4, description="Energía cognitiva (1=muy alta, 4=muy baja)")
1401     q2: int = Field(..., ge=1, le=4, description="Tensión física")
1402     q3: int = Field(..., ge=1, le=4, description="Rumiación / pensamientos recurrentes")
1403     q4: int = Field(..., ge=1, le=4, description="Activación negativa / nerviosismo")
1404     top_n: int = Field(default=3, ge=1, le=10)
1405     user_preferences: Optional[Dict[str, Any]] = None
1406     region_filter: Optional[str] = None # e.g. "Veracruz"
1407
1408
1409 class WellnessDestinationItem(BaseModel):
1410     id_destino: str
1411     nombre_lugar: str
1412     estado: str
1413     categoria_wellness: str
1414     match_pct: float
1415     beneficio_optimo_pct: float
1416     alineacion_pct: float
1417     wellness_sentiment_score: float
1418     rank: int
1419     nivel_aislamiento: float
1420     restauracion_pasiva: float
1421     demanda_fisica: float
1422     lat: Optional[float] = None
1423     lon: Optional[float] = None
1424     descripcion_bienestar: str
1425     beneficio_descripcion: str
1426
1427
1428 class WellnessAssessmentResponse(BaseModel):
1429     perfil_interno: str
1430     modo_viaje: str # 'modo_calma' | 'modo_restauracion' | 'modo_equilibrio'
1431     modo_viaje_label: str # 'Modo Calma' | 'Modo Restauración' | 'Modo Equilibrio'
1432     modo_viaje_descripcion: str
1433     confianza: float
1434     metodo: str
1435     destinations: List[WellnessDestinationItem]
1436
1437
1438 _wellness_destinations_cache: Optional[Any] = None
1439
1440
1441 def _get_wellness_destinations():
1442     global _wellness_destinations_cache
1443     if _wellness_destinations_cache is None:
1444         try:
1445             from wellness_matchmaker import load_destinations
1446             _wellness_destinations_cache = load_destinations()
1447         except Exception as e:
1448             logger.warning(f"[wellness] No se pudo cargar destinos: {e}")
1449             import pandas as pd
1450             _wellness_destinations_cache = pd.DataFrame()
1451     return _wellness_destinations_cache
1452
1453
1454 @app.post("/wellness/assess", response_model=WellnessAssessmentResponse)
1455 def wellness_assess(payload: WellnessAssessmentRequest):
1456     """
1457     Clasifica el perfil de vitalidad del usuario (Q1-Q4) y retorna
1458     las top-N recomendaciones de destinos wellness.

```

```

1459     Nunca expone el nombre técnico interno en modo_viaje.
1460     """
1461     try:
1462         from wellness_classifier import (
1463             get_classifier,
1464             MOD0_VIAJE_LABELS,
1465             MOD0_VIAJE_DESCRIPTION,
1466         )
1467         from wellness_matchmaker import recommend_wellness
1468
1469         clf = get_classifier()
1470         perfil, modo, proba_map, confianza, metodo = clf.predict(
1471             payload.q1, payload.q2, payload.q3, payload.q4
1472         )
1473
1474         destinations_df = _get_wellness_destinations()
1475         recs = recommend_wellness(
1476             destinations=destinations_df,
1477             perfil=perfil,
1478             q1=payload.q1,
1479             q2=payload.q2,
1480             q3=payload.q3,
1481             q4=payload.q4,
1482             top_n=payload.top_n,
1483             stress_confidence=confianza,
1484             user_preferences=payload.user_preferences,
1485             region_filter=payload.region_filter,
1486         )
1487
1488         return WellnessAssessmentResponse(
1489             perfil_interno=perfil,
1490             modo_viaje=modo,
1491             modo_viaje_label=MOD0_VIAJE_LABELS.get(modo, modo),
1492             modo_viaje_description=MOD0_VIAJE_DESCRIPTION.get(modo, ""),
1493             confianza=round(confianza, 3),
1494             metodo=metodo,
1495             destinations=[WellnessDestinationItem(**r) for r in recs],
1496         )
1497     except Exception as e:
1498         logger.error(f"[wellness] Error en assessment: {e}")
1499         raise HTTPException(status_code=500, detail=str(e))
1500
1501
1502 @app.get("/wellness/destinations")
1503 def wellness_destinations(
1504     estado: Optional[str] = Query(None),
1505     categoria: Optional[str] = Query(None),
1506     approved_only: bool = Query(True),
1507 ):
1508     """
1509     Lista destinos wellness disponibles en el catálogo.
1510     Si approved_only=True, solo retorna los aprobados por el admin.
1511     """
1512     try:
1513         df = _get_wellness_destinations()
1514         if df.empty:
1515             return {"destinations": []}
1516
1517         if estado:
1518             df = df[df.get("estado", "").str.lower() == estado.lower()]

```

```

1519         if categoria:
1520             cat_col = "categoria_wellness" if "categoria_wellness" in df.columns else "categoria_p
principal"
1521             df = df[df[cat_col].str.lower() == categoria.lower()]
1522             if approved_only and "wellness_status" in df.columns:
1523                 df = df[df["wellness_status"] == "approved"]
1524
1525             return {"destinations": df.to_dict(orient="records"), "total": len(df)}
1526     except Exception as e:
1527         raise HTTPException(status_code=500, detail=str(e))
1528
1529
1530 @app.get("/wellness/pending-count")
1531 def wellness_pending_count():
1532     """
1533     Conteo de servicios/POIs pendientes de validación wellness.
1534     Usada por AdminBadgesContext para el badge del sidebar.
1535     """
1536     try:
1537         import psycopg2
1538         conn = psycopg2.connect(
1539             host=os.getenv("POI_DB_HOST", "localhost"),
1540             port=int(os.getenv("POI_DB_PORT", 5432)),
1541             database=os.getenv("POI_DB_NAME", "smartur"),
1542             user=os.getenv("POI_DB_USER", "postgres"),
1543             password=os.getenv("POI_DB_PASSWORD", os.getenv("DB_PASSWORD", "")),
1544         )
1545         with conn.cursor() as cur:
1546             cur.execute(
1547                 "SELECT COUNT(*) FROM tourist_service WHERE wellness_status = 'pending'"
1548             )
1549             ts_count = cur.fetchone()[0]
1550             cur.execute(
1551                 "SELECT COUNT(*) FROM point_of_interest WHERE wellness_status = 'pending'"
1552             )
1553             poi_count = cur.fetchone()[0]
1554             conn.close()
1555             return {"total_pending": ts_count + poi_count, "services": ts_count, "pois": poi_count}
1556     except Exception as e:
1557         logger.warning(f"[wellness] pending-count DB error: {e}")
1558         return {"total_pending": 0, "services": 0, "pois": 0}
1559
1560
1561 @app.post("/wellness/train")
1562 def wellness_train(background_tasks: BackgroundTasks):
1563     """Re-entrena el clasificador de perfil wellness en background."""
1564     def _train():
1565         try:
1566             from wellness_classifier import WellnessProfileClassifier
1567             clf = WellnessProfileClassifier()
1568             metrics = clf.train()
1569             logger.info(f"[wellness-train] Completado: accuracy={metrics.get('accuracy', '?'):.3f}")
1570         except Exception as e:
1571             logger.error(f"[wellness-train] Error: {e}")
1572
1573     background_tasks.add_task(_train)
1574     return {"ok": True, "message": "Entrenamiento wellness iniciado en background"}
1575
1576

```

```
1577 @app.get("/wellness/metrics")
1578 def wellness_metrics():
1579     """Devuelve métricas del clasificador wellness guardadas en meta.json."""
1580     meta_path = Path("../models/wellness/wellness_profile.meta.json")
1581     if not meta_path.exists():
1582         raise HTTPException(
1583             status_code=404,
1584             detail="Modelo no entrenado aún. Ejecuta POST /wellness/train primero.",
1585         )
1586     with open(meta_path, encoding="utf-8") as f:
1587         meta = json.load(f)
1588     return {
1589         "classifier": {
1590             "accuracy": meta.get("accuracy"),
1591             "macro_f1": meta.get("macro_f1"),
1592             "classification_report": meta.get("classification_report", {}),
1593             "trained_at": meta.get("trained_at"),
1594             "n_samples": meta.get("n_samples"),
1595             "dataset": meta.get("dataset", "synthetic"),
1596         },
1597         "disclaimer": (
1598             "Métricas generadas sobre datos sintéticos (5,000 registros ATARAXIA). "
1599             "No reflejan desempeño con usuarios reales hasta acumular feedback de fit_rating."
1600         ),
1601     }
1602
1603
1604 if __name__ == "__main__":
1605     import uvicorn
1606     uvicorn.run(app, host="0.0.0.0", port=8000)
```